

Table des matières

Rapport d’audit de sécurité applicative — BAOU Finance

1. Avertissement sur la portée du présent document

2. Méthodologie et échelle d’évaluation

3. Vulnérabilités identifiées et corrigées

4. Risques résiduels identifiés — non corrigés

5. Tests de sécurité exécutés

6. Recommandations

7. Synthèse

1

1

1

3

5

6

7

7

Rapport d’audit de sécurité applicative — BAOU Finance

Produit	BAOU Finance — application mobile d’investissement BRVM
Périmètre	Backend Django/DRF, application mobile Flutter, portail d’administration React, infrastructure Docker
Version audité	v1.0.5
Date du rapport	29 août 2026
Auteur	Cissé Ahmed — Fondateur / Développeur, BAOU Finance
Nature	Audit interne (revue de code + tests automatisés)
Statut	Audit interne complet — pentest externe indépendant non réalisé

1. Avertissement sur la portée du présent document

Ce document est un **audit de sécurité interne**. Il ne constitue **pas** un rapport de test d’intrusion (pentest) au sens réglementaire.

La distinction est importante et doit être comprise par tout lecteur — partenaire, SGI, investisseur ou régulateur :

	Réalisé ici	Pentest au sens strict
Revue de code source	Oui	Généralement non (boîte noire)
Tests fonctionnels de sécurité automatisés	Oui	Oui
Test de charge / disponibilité	Oui	Partiel
Exploitation active de vulnérabilités	Non	Oui
Fuzzing, injection automatisée, scan de vulnérabilités	Non	Oui
Réalisé par un tiers indépendant	Non	Oui
Rapport opposable à un régulateur	Non	Oui

Conséquence directe : dans le cadre d’un dossier d’agrément BCEAO ou d’un partenariat avec une SGI agréée AMF-UMOA, ce document constitue une pièce de préparation et de bonne foi, mais **ne remplace pas** l’audit de sécurité indépendant qui sera exigé. Voir §6.

2. Méthodologie et échelle d’évaluation

2.1 Méthodes employées

1. **Revue de code statique** — lecture ligne à ligne des chemins sensibles : authentification, autorisation, mutations de solde, upload de fichiers, webhooks de paiement.

2. **Tests fonctionnels de sécurité automatisés** — scénarios d'attaque rejoués contre un backend réel (`test_api.py`), exécutés à chaque build en intégration continue.
3. **Tests unitaires de logique sensible** — verrouillage de compte (`test_lockout.py`), journal des mouvements de fonds (`test_ledger.py`).
4. **Test de charge** — k6, 20 utilisateurs virtuels simultanés (`load_test.js`).
5. **Revue de dépendances** — comparaison des versions épinglées avec les avis de sécurité publiés.

2.2 Échelle de risque

Classification inspirée de CVSS v3, simplifiée à quatre niveaux.

Niveau	Critère	Délai de correction attendu
Critique	Perte de fonds, accès non autorisé à des fonds ou usurpation de compte, exploitable sans authentification	Immédiat, bloque toute mise en production
Élevé	Fuite de données personnelles ou KYC, escalade de privilèges, contournement d'un contrôle réglementaire	Avant mise en production
Moyen	Dégradation de la sécurité sans exploitation directe, exposition facilitant une autre attaque	Avant ouverture au public
Faible	Défaut de configuration, dette technique sans exploitation connue	Planifié

2.3 Niveau d'évaluation par domaine

Le « niveau d'évaluation » indique la profondeur de vérification atteinte sur chaque domaine.

Domaine	Niveau d'évaluation	Justification
Authentification / session	Élevé	Revue de code + tests automatisés + test unitaire dédié
Autorisation / cloisonnement des données	Élevé	Revue de code + 6 scénarios d'attaque automatisés
Intégrité des mouvements de fonds	Élevé	Revue de code + tests + verrous DB vérifiés
Upload et stockage de documents KYC	Moyen	Revue de code + tests ; pas de fuzzing de fichiers malveillants
Intégration paiement (webhook Jèko)	Moyen	Revue de code + test de signature ; pas de test avec un attaquant actif
Sécurité infrastructure / réseau	Faible	Non testé — hors périmètre de cet audit interne
Sécurité applicative mobile (Flutter)	Faible	Revue de code seule ; pas d'analyse du binaire, pas de test sur appareil rooté

3. Vulnérabilités identifiées et corrigées

19 vulnérabilités ont été identifiées et corrigées au cours du développement. Chaque correctif dispose, sauf mention contraire, d'un test automatisé qui échoue si la faille réapparaît.

3.1 Niveau critique

#	Vulnérabilité	Impact	Correctif	Vérification
C-01	Le champ <code>userId</code> était ignoré lors d'une recharge administrateur — l'opération créditait le compte de l'administrateur connecté au lieu du compte client visé	Mouvement de fonds sur le mauvais compte	Prise en compte explicite du <code>userId</code> cible, réservée au rôle admin	<code>test_api.py</code> — vérifie que le crédit atterrit sur le compte client
C-02	Un client pouvait cibler un autre compte en fournissant un <code>userId</code> arbitraire à la création de transaction	Usurpation de compte, opérations sur les fonds d'autrui	Le <code>userId</code> d'un client est forcé à celui de sa session ; seul un admin peut le spécifier	<code>test_api.py</code> — « un client a pu cibler un autre compte ! »
C-03	Condition de course sur le solde — deux ordres simultanés pouvaient chacun passer le contrôle de provision et dépasser ensemble le solde réel (héritage du store Node en mémoire)	Découvert, création de monnaie	<code>SELECT FOR UPDATE</code> sur la ligne utilisateur dans un bloc atomique	Revue de code ; verrous vérifiés sur les 4 chemins de mutation
C-04	Deux ordres « pending » successifs pouvaient dépasser le solde — le débit n'intervenait qu'à la validation admin	Découvert	Gel du montant dès la création de l'ordre, remboursement automatique au rejet	<code>test_api.py</code> — séquence achat / rejet / vérification de solde
C-05	Montants stockés en flottant — l'erreur binaire s'accumulait à chaque opération (frais 0,5 % + TVA 18 % produisent de vraies fractions)	Divergence progressive des soldes, réconciliation impossible	Passage intégral en <code>Decimal(14,2)</code> ; contrat JSON inchangé	<code>test_api.py</code> — égalités exactes sur frais, TVA et solde
C-06	Mots de passe stockés en clair (comptes hérités du backend Node)	Compromission totale en cas de fuite de la base	Hachage <code>bcrypt</code> ; bascule silencieuse au premier login réussi	<code>test_api.py</code> — cycle login / mauvais mot de passe
C-07	Webhook de paiement Jèko non signé — un tiers pouvait forger une confirmation de dépôt	Création de fonds fictifs	Vérification HMAC-SHA256 en comparaison à temps constant (<code>compare_digest</code>)	<code>test_api.py</code> — webhook signé rejoué de bout en bout

3.2 Niveau élevé

#	Vulnérabilité	Impact	Correctif	Vérification
H-01	Le répertoire /uploads/ était public — pièces d'identité, selfies et justificatifs de domicile accessibles sans aucune authentification	Fuite massive de données KYC	Route protégée : seul le propriétaire (préfixe userId du nom de fichier) ou un admin peut lire	test_api.py — « document KYC lisible sans authentification ! » (401 attendu)
H-02	Auto-validation du KYC par le client via le champ kycStatus de la mise à jour de profil	Contournement du contrôle réglementaire d'entrée en relation	Le statut KYC n'est modifiable que par un admin, sur une route dédiée	test_api.py — le statut reste pending après tentative
H-03	Statut KYC non validé — toute chaîne était acceptée : une faute de frappe verrouillait le client en silence, une chaîne trop longue faisait planter PostgreSQL	Déni de service, blocage client irréversible	Liste blanche KYC_STATUSES + validation de longueur	test_api.py — 400 sur "verifie" et sur 40 caractères
H-04	Absence de limitation sur les endpoints d'authentification — le throttle global à 300/min laissait tout le loisir de brute-forcer un mot de passe	Compromission de compte par force brute	Throttle dédié AuthThrottle à 10/min sur login et réinitialisation de mot de passe	Observé en test de charge (rejets 429 conformes)
H-05	Absence de verrouillage progressif de compte	Force brute distribuée dans le temps	Paliers : 5 échecs → 10 min, 10 → 1 h, 15 → réinitialisation obligatoire	test_lockout.py — les 3 paliers vérifiés
H-06	Secret JWT par défaut en dur dans le code	Forge de jetons d'authentification arbitraires	JWT_SECRET obligatoire ; démarrage refusé en production s'il est absent	Revue de code ; échec au démarrage vérifié
H-07	SECRET_KEY Django par défaut en dur	Compromission des signatures Django	Dérivée de JWT_SECRET, lui-même obligatoire et aléatoire	Revue de code
H-08	Django 5.1.4 — version affectée par plusieurs CVE publiées entre juin et décembre 2025, dont une injection SQL de sévérité « haute » dans FilteredRelation	Injection SQL potentielle	Montée en version 5.1.15	requirements.txt

3.3 Niveau moyen

#	Vulnérabilité	Impact	Correctif	Vérification
M-01	Upload sans contrôle de type ni de taille	Stockage de contenu arbitraire, saturation disque	Vérification des octets d'en-tête (magic bytes JPEG/PNG), limite 15 Mo, nom de fichier assaini	test_api.py — 400 sur base64 invalide, 400 sur non-image

#	Vulnérabilité	Impact	Correctif	Vérification
M-02	CORS non restreint	Requêtes cross-origin depuis un domaine tiers	CORS_ALLOWED_ORIGINS explicite ; avertissement au démarrage si vide en production	Revue de code
M-03	Absence de journal des mouvements de fonds — aucun enregistrement immuable des variations de solde	Impossibilité d'auditer ou de réconcilier ; non-conformité aux exigences de traçabilité BCEAO / AMF-UMOA	Modèle LedgerEntry en ajout seul (modification refusée, clés étrangères en PROTECT) ; apply_balance() établi comme point de mutation unique	test_ledger.py — vérifie qu'une seule affectation de solde subsiste dans le code, et le chaînage solde avant / après

3.4 Niveau faible

#	Vulnérabilité	Impact	Correctif
L-01	Dépendance @google/genai non utilisée dans le portail d'administration, avec un .env.example réclamant une clé d'API Gemini	Surface d'attaque inutile ; invite à renseigner une clé sans usage	Dépendance et fichier retirés

4. Risques résiduels identifiés — non corrigés

Ces points sont identifiés, documentés, et **non traités à ce jour**. Ils sont listés ici par transparence.

#	Risque	Niveau	Description	Recommandation
R-01	Déconnexion sans révocation	Moyen	L'endpoint /api/auth/logout renvoie un succès mais ne révoque rien côté serveur. Un jeton JWT dérobé reste valide jusqu'à son expiration naturelle (24 h), y compris après déconnexion de l'utilisateur légitime.	Liste de révocation (identifiant jti en base ou cache) consultée à chaque requête authentifiée
R-02	ALLOWED_HOSTS = ["*"]	Faible	Le filtrage d'en-tête Host est délégué à l'infrastructure amont (ngrok / Railway). Correct dans l'architecture actuelle, dangereux si le backend est un jour exposé directement.	Restreindre lors du passage sur une infrastructure de production maîtrisée
R-03	Jeton stocké en clair côté mobile	Moyen	Le jeton d'authentification est conservé en mémoire et l'URL de l'API dans SharedPreferences, non chiffré. Exploitable sur un appareil rooté ou compromis.	Migrer vers flutter_secure_storage (Keychain / Keystore)

#	Risque	Niveau	Description	Recommandation
R-04	Jeton admin en localStorage	Moyen	Le portail d'administration stocke son jeton en localStorage, accessible à tout script injecté. Aucune faille XSS n'a été trouvée (aucun dangerouslySetInnerHTML, aucun eval), mais la défense en profondeur fait défaut.	Cookie httpOnly + SameSite=Strict, ou en mémoire seule
R-05	Pas de rotation des jetons	Faible	Durée de vie fixe de 24 h, sans jeton de rafraîchissement.	Couple jeton court / jeton de rafraîchissement lors du passage à l'échelle
R-06	Absence de plan de continuité d'activité formalisé	Élevé (réglementaire)	Une sauvegarde automatisée de la base existe (conteneur db_backup), mais aucune procédure de restauration documentée ni testée. Exigence explicite de la BCEAO.	Documenter et tester une restauration complète
R-07	Journalisation applicative rudimentaire	Moyen	Les erreurs d'intégration paiement sont tracées par print vers la sortie standard. Pas de journal centralisé, pas de conservation, pas d'alerte.	Journalisation structurée + conservation conforme aux exigences de supervision
R-08	Aucun test d'intrusion externe	Élevé	Voir §1 et §6.	Mandater un prestataire indépendant avant l'ouverture au public

5. Tests de sécurité exécutés

Test	Nature	Couverture	Fréquence
test_api.py	Scénarios d'attaque de bout en bout contre un backend réel	Authentification, cloisonnement des données, verrou KYC, provision, entrées invalides, usurpation, protection des documents, webhook signé	À chaque build (CI)
test_lockdown.py	Test unitaire	Paliers de verrouillage de compte (5 / 10 / 15 échecs)	À chaque build (CI)
test_ledger.py	Test unitaire + test d'architecture	Point de mutation unique du solde, chaînage du journal, plancher à zéro	Manuel
api/brvm.py	Validation du parseur de données de marché	Robustesse du chargement des cotations	À chaque build (CI)
load_test(k6)	Charge — 20 utilisateurs virtuels, 70 s	Disponibilité et latence sous charge	Manuel

Scénarios d'attaque couverts par test_api.py — chacun échoue si la protection disparaît :

- Accès à un endpoint admin avec un jeton client → 403 attendu
- Accès sans jeton et avec un jeton forgé → 401 attendu
- Lecture d'un document KYC sans authentification → 401 attendu
- Lecture du document KYC d'un tiers → 403 attendu
- Auto-validation du statut KYC par un client → statut inchangé attendu
- Opération d'argent avec un KYC non vérifié → 403 attendu
- Achat sans provision suffisante → 400 attendu
- Vente de titres non détenus → 400 attendu
- Quantité non numérique ou négative → 400 attendu (et non 500)
- Ciblage d'un autre compte par un client → opération rabattue sur son propre compte
- Statut KYC invalide ou surdimensionné → 400 attendu
- Lecture des transactions d'autrui → cloisonnement vérifié

Résultat du test de charge (20 VU, 70 s, throttle désactivé pour mesurer la capacité brute) : 100 % de succès, latence p95 = 73 ms, aucune erreur. Avec le throttle de production actif, 76 % des requêtes sont rejetées en 429 — comportement attendu et conforme, la charge de test provenant d'une adresse IP unique.

6. Recommandations

6.1 Avant toute ouverture au public — bloquant

1. **Mandater un test d'intrusion externe indépendant.** C'est la lacune principale de ce dossier. Aucune revue interne, aussi rigoureuse soit-elle, ne remplace un tiers qui attaque réellement le système. Périmètre minimal recommandé : API backend en boîte grise, application mobile (analyse du binaire, interception du trafic), portail d'administration.
2. **Corriger R-01** (révocation de jeton à la déconnexion) — faible coût, risque réel.
3. **Documenter et tester une restauration de sauvegarde** (R-06) — exigence réglementaire explicite.

6.2 Avant le dossier d'agrément ou le partenariat SGI

4. Migrer le stockage des jetons vers un stockage sécurisé (R-03, R-04).
5. Mettre en place une journalisation structurée et conservée (R-07).
6. Formaliser le plan de continuité d'activité et la politique de gestion des incidents.
7. Faire réviser la répartition des obligations KYC/AML avec la SGI partenaire : dans un montage où BAOU est le canal digital d'une SGI agréée, la responsabilité réglementaire de l'entrée en relation revient normalement à la SGI, le module KYC de BAOU devenant une collecte déléguée. Ce point doit figurer explicitement dans la convention de partenariat.

6.3 Maintenance continue

8. Surveiller les avis de sécurité Django et remonter de version sans attendre (la version 5.1.4 initialement embarquée accusait plus de six mois de correctifs de retard).
9. Réexécuter la suite de tests de sécurité à chaque déploiement — déjà en place en intégration continue.
10. Reconduire un audit complet à chaque évolution majeure du parcours de fonds.

7. Synthèse

Indicateur	Valeur
Vulnérabilités corrigées	19
dont critiques	7
dont élevées	8
dont moyennes	3
dont faibles	1
Risques résiduels documentés	8
dont élevés	2 (dont l'absence de pentest externe)

Indicateur	Valeur
Scénarios d’attaque automatisés	12
Couverture par tests automatisés	Authentification, autorisation, mouvements de fonds, KYC, upload, webhook de paiement

Appréciation générale. Les chemins critiques — authentification, cloisonnement des données, intégrité des mouvements de fonds — ont été audités en profondeur et sont protégés par des tests automatisés qui échouent si une protection disparaît. Les vulnérabilités les plus graves identifiées relevaient toutes de la logique métier (usurpation de compte, condition de course sur le solde, contournement du KYC) plutôt que de failles techniques classiques, ce qui est cohérent avec le profil d’une application financière.

Les deux faiblesses structurelles restantes sont l’**absence de test d’intrusion indépendant** et l’**absence de plan de continuité testé**. Aucune des deux ne relève du code : elles relèvent de l’organisation, et devront être traitées avant toute mise en production réelle ou tout dépôt de dossier réglementaire.

Document interne. Rédigé de bonne foi sur la base des éléments techniques vérifiables au 29 août 2026. Ne constitue ni une certification, ni un avis juridique, ni un rapport de test d’intrusion.